

Report 3: Prompt Engineering & Context Injection Analysis

Document Classification: Technical Research Report **Date:** February 17, 2026 **Scope:** How modifying the instruction prompt can improve VSP-LLM accuracy on real-world data **Current Performance Baseline:** Mean WER 67.0% on english_1k (860 segments)

1. Executive Summary

VSP-LLM uses a simple, fixed instruction prompt: "Recognize this speech in English. Input : " followed by embedded visual features. This prompt provides **zero context** about the video content, speaker, domain, or expected vocabulary.

Because the model's decoder is LLaMA-2-7B — a full LLM trained on trillions of tokens — it is inherently capable of instruction-following and context-aware generation. The current prompt wastes this capability entirely. By injecting contextual information into the prompt, we can **guide the LLM's language prior toward the correct domain**, reducing the ambiguity that causes hallucination.

This report analyzes 7 prompt engineering strategies, ranks them by expected impact and implementation difficulty, and provides exact code modifications for each.

Estimated impact: The most promising strategies (topic context + word count hints) could reduce WER by 5-15 percentage points on segments where contextual information is available, with the strongest effects on the currently-hallucinated segments.

2. How the Current Prompt Works

2.1 Architecture Context

The VSP-LLM model processes input in this order:

1. Instruction text → LLaMA tokenizer → Embedding layer → `instruction_embedding`
2. Video frames → AV-HuBERT encoder → Linear projection → `reduced_enc_out`
3. Concatenation: [`instruction_embedding` | `reduced_enc_out`]
4. LLaMA-2 generates text conditioned on this concatenated input

From `vsp_llm.py` (generate method, lines 381-396):

```

instruction = kwargs['source']['text']
instruction_embedding = self.decoder.model.model.embed_tokens(instruction)
llm_input = torch.cat((instruction_embedding, reduced_enc_out), dim=1)

outputs = self.decoder.generate(inputs_embeds=llm_input,
                                num_beams=num_beams,
                                max_new_tokens=max_length,
                                ...)

```

The instruction tokens and visual features occupy the **same embedding space** — they're concatenated and processed together by LLaMA. This means **any text prepended to the visual features is treated as context** by the LLM's attention mechanism.

2.2 Current Instruction

The paper defines two instructions (Section 3.3): - **VSR**:

"Recognize this speech in English. Input : " - **VST**:

"Translate this English speech to {TARGET_LANG}. Input : "

The model was **trained** with these exact instructions. Modifying the instruction at inference time may or may not work — it depends on how much the model generalized from instruction-following during pre-training (LLaMA-2) vs. how rigidly it was fine-tuned to expect the exact training prompt (QLoRA).

2.3 Key Insight: LLaMA-2's Instruction-Following Capability

LLaMA-2 was pre-trained on 2 trillion tokens and has strong instruction-following abilities. Even after QLoRA fine-tuning for VSP-LLM, the base model retains its understanding of natural language instructions. This means: 1. **Additional context in the prompt IS processed** by the model 2. **The model can use contextual clues** to disambiguate visual features 3. **But** — significant prompt deviation from the training format may confuse the fine-tuned layers

3. Prompt Engineering Strategies

3.1 Strategy 1: Topic/Domain Context

Difficulty: Low (metadata often available) **Expected impact:** High (5-10 WER points on applicable segments)

Concept

Add a topic description before the recognition instruction to prime the LLM's vocabulary distribution.

Proposed Prompts

```
# Option A: Topic hint
"The speaker is discussing {TOPIC}. Recognize this speech in English. Input : "

# Option B: Domain context
"This is a {DOMAIN} video. Recognize this speech in English. Input : "

# Option C: Detailed context
"The speaker is giving a {DOMAIN} presentation about {TOPIC}.
Recognize this speech in English. Input : "
```

Examples from the Data

Segment	Current HYP (wrong)	Reference	Useful Context
-0dKADuskCI_6	"because we have that cause and effect correlation..."	"then we have the costal cartilages costal always refers to ribs"	Topic: "anatomy"
0HTUgySq5ko_0	"was like you have three months of wear here..."	"now is you have your vpn software here and it's encrypted"	Topic: "cybersecurity"
14hw7NuQV1s_7	"the cheese into five one four eight two"	"this is the g0514x2b"	Topic: "product review, model number"

In each case, knowing the topic would dramatically shift the LLM's vocabulary prior toward the correct domain. "Costal cartilages" becomes plausible when the model knows it's an anatomy lecture. "VPN software encrypted" becomes plausible in a cybersecurity context.

How to Obtain Topic Information

1. **Video metadata:** YouTube title, description, tags (available at scrape time)
2. **First-pass ASR:** Run Whisper on the audio to get a topic summary, then use it as context for the visual-only pass
3. **Manual labeling:** For high-value content, annotate the topic
4. **LLM classification:** Feed the first few seconds of Whisper transcript to an LLM to classify the domain

Implementation

In `vsp_llm_decode.py`, the instruction is loaded from the dataset:

```
# Current code (line 259):
instruction = tokenizer.decode(
```

```
sample['net_input']['source']['text'][i].int().cpu(),
skip_special_tokens=True, clean_up_tokenization_spaces=False
)
```

To inject context, modify the instruction before encoding in the dataset loader
(`vsp_llm_dataset.py`) or at decode time:

```
# Modified approach – inject topic context
topic = get_topic_for_segment(sample['utt_id'][i]) # From metadata file
base_instruction = "Recognize this speech in English. Input : "
if topic:
    instruction_text = f"The speaker is discussing {topic}. {base_instruction}"
else:
    instruction_text = base_instruction

# Re-tokenize the modified instruction
instruction_tokens = tokenizer.encode(instruction_text, return_tensors='pt')
```

Caveat: This requires modifying how instructions are embedded. The current pipeline pre-tokenizes instructions in the dataset. A cleaner approach is to add a `--context` argument to the decode script that prepends text to every instruction.

3.2 Strategy 2: Word Count Hints

Difficulty: Low (derivable from video duration) **Expected impact:** Medium (3-7 WER points)

Concept

Tell the model approximately how many words to expect, reducing verbose hallucination.

Proposed Prompts

```
# Option A: Approximate count
"The speaker says approximately {N} words. Recognize this speech in English. Input : "

# Option B: Range
"The speaker says between {N-2} and {N+2} words. Recognize this speech in English. Input : "

# Option C: Duration-based
"This is a {DURATION}-second clip. Recognize this speech in English. Input : "
```

Derivation

Average English speech rate: ~2.5-3.0 words/second. For a segment of known duration: - **5-second clip:** expect ~13-15 words - **10-second clip:** expect ~25-30 words - **3-second clip:** expect ~8-9 words

Segment duration is **always known** from the filename (e.g., `_00_000000_000300` = 0.0s to 3.0s at 100fps equivalent, or derivable from frame count at 25fps).

Evidence This Could Help

Our data shows the model often generates text of wildly inappropriate length: - Reference: 2 words → Hypothesis: 6 words (200% WER) - Reference: 14 words → Hypothesis: 18 words (121% WER) - Reference: 7 words → Hypothesis: 0 words (100% WER)

A word count hint would constrain the LLM's generation length, addressing the insertion problem (corpus WER 125.5%).

Implementation

```
# Estimate expected word count from segment duration
fps = 25
start_frame, end_frame = parse_segment_timing(utt_id)
duration_sec = (end_frame - start_frame) / fps
expected_words = int(duration_sec * 2.7) # avg speech rate

instruction = f"The speaker says approximately {expected_words} words. Recognize this speech"
```

Risk: If the word count estimate is wrong (speaker pauses, talks fast/slow), the hint could constrain the model incorrectly. Using a range ("10-20 words") mitigates this.

3.3 Strategy 3: Vocabulary Constraints / Word Lists

Difficulty: Medium (requires domain knowledge) **Expected impact:** High for named entities (5-10 NEA F1 points)

Concept

Provide a list of expected vocabulary words — especially domain-specific terms — so the model can select from them rather than hallucinating alternatives.

Proposed Prompts

```
# Option A: Vocabulary list
"The following words may appear: {WORD_LIST}. Recognize this speech in English. Input : "

# Option B: Named entities
"Names and terms that may appear: {ENTITY_LIST}. Recognize this speech in English. Input : "
```

Why This Matters

Named entity accuracy (NEA F1) is only 38.8%. The model misses domain-specific terms because they're rare in its training data. By providing candidate words, we shift the LLM's probability distribution toward the correct terms.

Examples

Reference	Missed Entity	Useful Vocabulary Hint
"costal cartilages"	costal, cartilages	"anatomy terms: costal, cartilage, ribs, thorax"
"vpn software encrypted"	vpn, software, encrypted	"tech terms: VPN, encryption, software"
"g0514x2b"	g0514x2b	"model number: G0514X2B"
"samsung"	samsung	"brands: Samsung"
"beatniks in chicago"	beatniks, chicago	"places: Chicago; venues: Beatniks"
"orangina"	orangina	"beverages: Orangina"

How to Obtain Vocabulary Lists

1. **From Whisper ASR:** Extract named entities from the audio transcription
2. **From video metadata:** YouTube title often contains key terms
3. **Domain dictionaries:** Pre-built vocabulary lists for common domains (medical, tech, cooking)
4. **User-provided glossary:** Let users specify expected terminology

Implementation

```
# Build vocabulary hint from Whisper transcription's named entities
whisper_transcript = load_whisper_transcript(segment_id)
entities = extract_named_entities(whisper_transcript) # Using spaCy

if entities:
    entity_str = ", ".join(entities)
    instruction = f"The following terms may appear: {entity_str}. Recognize this speech in
```

Caveat: If using Whisper ASR to provide vocabulary hints, you're partly circular (using audio-based ASR to help visual-only ASR). This is valid for scenarios where: (a) the goal is lip-reading-specific accuracy, (b) Whisper is unavailable at deployment but vocabulary can be pre-computed, or (c) Whisper provides noisy/partial results that can still hint vocabulary.

3.4 Strategy 4: Speaker Description

Difficulty: Medium (requires speaker metadata) **Expected impact:** Low-Medium (2-5 WER points)

Concept

Describe the speaker's characteristics to help the model resolve ambiguities related to accent, speaking style, and vocabulary level.

Proposed Prompts

```
"The speaker is a {DESCRIPTION}. Recognize this speech in English. Input : "
```

```
# Examples:
```

```
"The speaker is a male professor giving a lecture."
```

```
"The speaker is a female product reviewer."
```

```
"The speaker is a male chef demonstrating a recipe."
```

Rationale

Different speakers use different vocabulary patterns. A professor is likely to use academic language; a chef is likely to mention ingredients and techniques. This contextual information helps the LLM select from the right vocabulary distribution.

Implementation Complexity

Speaker descriptions require video-level metadata — typically available from YouTube channel information or manual annotation. Less scalable than topic-based context but potentially more useful for specialized applications.

3.5 Strategy 5: Multi-Turn Refinement

Difficulty: High (requires pipeline modification) **Expected impact:** Medium-High (5-10 WER points)

Concept

Run the model twice: 1. **First pass:** Standard recognition → rough hypothesis 2. **Second pass:** Feed the rough hypothesis back as context → refined hypothesis

Proposed Two-Pass Prompt

```
# Pass 1 (standard):
"Recognize this speech in English. Input : {visual_features}"
→ rough_hyp: "because we have that cause and effect correlation"

# Pass 2 (refinement):
"A previous attempt at recognizing this speech produced: '{rough_hyp}'.
Please refine this transcription by correcting any errors.
Recognize this speech in English. Input : {visual_features}"
```

Why This Could Work

The first pass captures the general structure and some correct words. The second pass uses this as a **language model prior** — the LLM can identify implausible phrases in the rough hypothesis and correct them while attending to the same visual features.

This is analogous to: - **Deliberation networks** in ASR (Xia et al., 2017) - **Iterative refinement** in NMT (Lee et al., 2018) - **Self-consistency** in LLM reasoning (Wang et al., 2023)

Implementation

```
# Two-pass decode
# Pass 1: Standard
rough_hyp = model.generate(instruction="Recognize this speech in English. Input : ",
                           visual_features=features)

# Pass 2: Refinement
refined_instruction = (
    f"A lip-reading model produced this transcription: '{rough_hyp}'. "
    f"Review and correct it based on the lip movements. "
    f"Recognize this speech in English. Input : "
)
refined_hyp = model.generate(instruction=refined_instruction,
                             visual_features=features)
```

Risk: The model may simply reproduce the first-pass output without correction. The fine-tuning was not designed for refinement instructions, so the model may not know how to "correct" text. This strategy is the most speculative.

Cost: Doubles inference time (two full forward passes per segment).

3.6 Strategy 6: Negative Prompting / Anti-Hallucination Instructions

Difficulty: Low **Expected impact:** Low-Medium (2-5 WER points)

Concept

Instruct the model to avoid common failure modes.

Proposed Prompts

```
# Option A: Conciseness instruction
"Recognize this speech in English. Output only the words spoken, nothing more. Input : "

# Option B: Anti-hallucination
"Recognize this speech in English. If you are unsure, output only the words you are
confident about. Do not guess. Input : "

# Option C: Faithfulness instruction
"Carefully recognize this speech in English. Only output words that match the lip
movements. Do not add filler or context. Input : "
```

Rationale

LLaMA-2 has been trained on instruction-following tasks and understands directives like "only", "do not", "nothing more". These constraints may reduce the model's tendency to generate verbose, fabricated text.

Caveat

The model was fine-tuned with "Recognize this speech in English. Input : " — significant deviation from this format may degrade the fine-tuned visual-to-text mapping. The QLoRA adapters learned to associate this specific prompt structure with the lip-reading task. Adding too many extra instructions may push the model out of its fine-tuned distribution.

Recommendation: Test with minimal additions first ("Output only the words spoken.") before trying longer anti-hallucination prompts.

3.7 Strategy 7: Language/Format Constraints

Difficulty: Low **Expected impact:** Low (1-3 WER points)

Concept

Specify expected output format to prevent structural hallucination.

Proposed Prompts

```
# Option A: Lowercase constraint
"Recognize this speech in English. Output in lowercase without punctuation. Input : "
```

```
# Option B: Single sentence
```

```
"Recognize this speech as a single English sentence. Input : "
```

This is minor but can help with post-processing consistency.

4. Strategy Comparison Matrix

Strategy	Expected WER Impact	NEA Impact	Difficulty	Requires Metadata	Doubles Compute
1. Topic context	-5 to -10 pts	+5-10 F1	Low	Yes (topic)	No
2. Word count hints	-3 to -7 pts	Minimal	Low	No (from duration)	No
3. Vocabulary lists	-3 to -5 pts	+10-15 F1	Medium	Yes (entities)	No
4. Speaker description	-2 to -5 pts	Minimal	Medium	Yes (speaker)	No
5. Multi-turn refinement	-5 to -10 pts	+5 F1	High	No	Yes
6. Anti-hallucination	-2 to -5 pts	Minimal	Low	No	No
7. Format constraints	-1 to -3 pts	Minimal	Low	No	No

5. Critical Caveat: Training Distribution Mismatch

All prompt engineering strategies face a fundamental limitation: **the model was fine-tuned with a fixed prompt format**. The QLoRA adapters (rank 16, applied to LLaMA-2) learned to associate specific instruction embeddings with the lip-reading task.

What This Means in Practice

- **Small additions** to the prompt (topic hints, word counts) are likely to work because LLaMA-2's base instruction-following capability is preserved through QLoRA
- **Large deviations** from the training prompt may confuse the fine-tuned layers, causing the model to treat the input as a general text completion task rather than a lip-reading task

- **The visual features remain fixed** — prompt changes only affect how the LLM *interprets* the visual embeddings, not the embeddings themselves

Recommendation for Testing

Test prompt modifications on a small subset (50-100 segments) before running on the full dataset. Compare: 1. Does the model still produce speech-like output? (Sanity check — if prompt changes cause the model to output non-speech text like "I'll help you with that...", the prompt has broken the fine-tuning) 2. Does WER improve on average? (Aggregate metric) 3. Does hallucination rate decrease? (% segments with WER $\geq 100\%$)

6. Implementation Roadmap

Phase 1: Low-Hanging Fruit (1-2 days)

1. **Word count hint from duration** (Strategy 2)
2. No metadata required
3. Derivable from segment filename
4. Modify instruction string in decode script
5. **Anti-hallucination instruction** (Strategy 6)
6. Add "Output only the words spoken." to prompt
7. Zero additional data required

Phase 2: Context Injection (3-5 days)

1. **Topic context from Whisper transcript** (Strategy 1)
2. Run LLM classification on Whisper transcripts to extract topics
3. Build topic metadata file
4. Inject into instruction
5. **Vocabulary hints from Whisper named entities** (Strategy 3)
6. Extract entities from Whisper transcript using spaCy
7. Build per-segment entity lists
8. Inject into instruction

Phase 3: Advanced (1-2 weeks)

1. **Multi-turn refinement** (Strategy 5)

2. Modify decode pipeline for two-pass inference
 3. Evaluate compute vs. accuracy trade-off
-

7. References

1. Yeo et al. (2024). "VSP-LLM: Visual Speech Processing with LLMs." arXiv:2402.15151.
2. Touvron et al. (2023). "LLaMA 2: Open Foundation and Fine-Tuned Chat Models." arXiv:2307.09288.
3. Dettmers et al. (2023). "QLoRA: Efficient Finetuning of Quantized Language Models." arXiv:2305.14314.
4. Wang et al. (2023). "Self-Consistency Improves Chain of Thought Reasoning in Language Models." ICLR 2023.
5. Xia et al. (2017). "Deliberation Networks: Sequence Generation Beyond One-Pass Decoding." NeurIPS 2017.
6. Lee et al. (2018). "Deterministic Non-Autoregressive Neural Sequence Modeling by Iterative Refinement." EMNLP 2018.
7. Wei et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." NeurIPS 2022.
8. Brown et al. (2020). "Language Models are Few-Shot Learners." NeurIPS 2020. — In-context learning and prompt engineering foundations.